

eCetatean

Tanulasi Utmutato

Egy 48 oras hackathon projekt teljes megertese

Tartalomjegyzek:

1. Hogyan mukodik egy modern webalkalmazas?
2. TypeScript alapok
3. React 19 es komponensek
4. UI retteg: Tailwind CSS es shadcn/ui
5. Backend: Hono.js es REST API-k
6. Adatbazis: Supabase es PostgreSQL
7. Autentikacio
8. AI integracios
9. PDF generalas
10. Biztonsag es audit
11. Deployment
12. Osszefoglalas es kovetkezo lepesek

1. fejezet: Hogyan mukodik egy modern webalkalmazas?

Mielott belemerulnenk a kodba, ertsuk meg az alapokat. Egy modern webalkalmazas harom nagy reszbol all: a frontend (amit a felhasznalo lat), a backend (amit a szerver vegez), es az adatbazis (ahol az adatok tartosan tarolodnak). Az eCetatean projekt mindharem komponenst használja, es a fejlesztok tudatosan választottak el ezeket egymastól.

1.1 Kliens-szerver modell es a HTTP kerelem-valasz ciklus

Kepzelj el egy ettermet. A vendeg (kliens = bongeszod) rendel valamit, a pincer (HTTP protokoll) továbbítja a konyhanak (szerver), ahol elkészül az étel (válasz), majd visszakerül a vendeghez. Az internet PONTOSAN így működik.

A HTTP (HyperText Transfer Protocol) egy szovegalapU protokoll, amellyel a bongeszó és a szerver kommunikál. Minden HTTP kerelem tartalmaz:

- Metodus: GET (lekerdezes), POST (kuldendo adat), PUT/PATCH (frissites), DELETE (torles)
- URL: hova kuldjuk a keremet (pl. <https://api.ecetatean.ro/api/profile>)
- Header-ok: plusz informaciok (Authorization, Content-Type stb.)
- Body (teszt): az elkuldott adatok (pl. JSON formaban)

A valasz tartalmaz egy statusz kodot (200 = OK, 201 = Letrehozva, 400 = Hibas kerelem, 401 = Nem autentikalt, 404 = Nem talalhato, 500 = Szerver hiba) es egy body-t (a valasz adata).

1.2 Mi a JSON es miért használják az API-k?

A JSON (JavaScript Object Notation) egy emberi szemmel is olvasható adatformat. Az eCetatean backend minden API választ JSON-ban kuldoz vissza. Peldaul:

```
{
  "success": true,
  "data": {
    "full_name": "Kovacs Janos",
    "cnp": "1850312123456",
    "city": "Cluj-Napoca",
    "email": "kovacs@example.com"
  }
}
```

A JSON elonyei: konnyen parszealhato minden programozasi nyelvben, ember által olvasható, kicsi a merete, es a JavaScript nativan támogatja (hiszen onnan származik a nev is).

1.3 Miért vannak elvalasztva a frontend es a backend?

Az eCetatean projektben a frontend es a backend ket kulonallo alkalmazas, kulonbozo helyen telepítve. Ennek több oka van:

- Kulonbozo deployment helyek: A frontend Cloudflare Workers-en fut (globalis CDN), a backend Vercel-en (serverless). Így a frontend gyorsan betöltődik barhan a vilagon.

- Különbozo aggodalmak (separation of concerns): A frontend a megjelenitessel foglalkozik (UI), a backend az uzleti logikával. Ha ujratervezed a gombot, nem kell hozzanyulni az adatbazishoz.
- Skalazhatasag: Ha sok felhasznalo van, a backendbol tobb peldanyt indithatunk el anelkul, hogy a frontendhez nyulnank.
- Biztonsag: Az adatbazis tikos kulcsok csak a backenden talalhatok, soha nem kerul a bongeszohoz.

1.4 Az eCetatean architektura

Frontend	React 19 + TanStack Router, Cloudflare Workers-en (wrangler deploy)
Backend	Hono.js + TypeScript, Vercel serverless functions-on
Adatbazis	Supabase PostgreSQL (PostgREST API + Auth + Storage)
AI	Anthropic Claude Sonnet 4.6 (elsobbseg) vagy Google Gemini 2.0 Flash (backup)
PDF	@pdfme/pdf-lib + pdf-lib: PDF kitoltes es generalas

Az adatfolyam a kovetkezo: a bongeszó HTTP kerelmet kuldoz a Cloudflare Workers-en futó React alkalmazasnak, amely megjeleniti a UI-t. Amikor adatra van szukseg, a React fetch()-hivas utjan kommunikal a Vercel-en futó Hono.js backenddel, amely Supabase-t hasznal az adatok tarolasara es lekeresere.

1.5 A tenyleges fetch hivas a frontendben

Lassuk meg a frontend/src/lib/api.ts fajlban, hogyan zajlik egy tenyleges HTTP kerelem:

```
// frontend/src/lib/api.ts
async function request<T>({
  path: string,
  init: RequestInit,
  getToken: GetToken,
}): Promise<T> {
  const base = getApiBaseUrl(); // pl. https://api.ecetatean.ro
  const url = `${base}${path}`; // pl. https://api.ecetatean.ro/api/profile

  const token = await getToken(); // JWT token lekerdezese Supabase Auth-tól
  const headers = new Headers(init.headers);
  if (token) headers.set("Authorization", `Bearer ${token}`);
  // Minden kereshez hozzáadjuk: Authorization: Bearer eyJhbGci...

  let response: Response;
  try {
    response = await fetch(url, { ...init, headers });
    // fetch() = beépített böngésző API, HTTP keremet küld
  } catch (err) {
    throw buildFetchError(err); // Halozati hiba kezelése
  }

  // Ha PDF-et varunk, külön kezeljük
  if (contentType.includes("application/pdf")) {
    return (await response.blob()) as unknown as T;
  }

  // Alap eset: JSON válasz parszélése
  body = await response.json() as Envelope<T>;
  if (!body || body.success === false) {
    throw new ApiError(response.status, body.error);
  }
  return body.data; // Csak a "data" mezőt adjuk vissza
}
```

Figyeld meg: minden kereshez automatikusan hozzáadjuk a JWT token az Authorization headerben. Így a backend tudja ki küldötte a keremet. Az Envelope típus azt jelenti, hogy a backend mindig { success: true, data: T } vagy { success: false, error: string } formátban válaszol.

1.6 Egy valódi HTTP endpoint a backenden

Most nezzük meg a backend/src/routes/profile.ts fájlban, hogyan válaszol a szerver a GET /api/profile keresre:

```
// backend/src/routes/profile.ts
profileRoute.get('/', requireAuth, async (c) => {
  const userId = c.get('userId')
  // requireAuth middleware már ellenőrizte a JWT token-t
  // és beállította a userId-t a kontextusban

  const { data, error } = await supabaseAdmin
    .from('profiles')           // 'profiles' tábla kiválasztása
    .select('full_name, cnp, ...') // Csak a szükséges mezők
    .eq('user_id', userId)      // WHERE user_id = ?
    .maybeSingle()             // Max 1 sort várunk vissza

  if (error) {
    return c.json({ success: false, error: 'Hiba...' }, 500)
  }

  return c.json({ success: true, data: data ?? null })
  // 200 OK HTTP státusz + JSON válasz
})
```

Az egész folyamat: (1) Bővebb GET /api/profile küldése JWT-vel, (2) requireAuth middleware ellenőrzése a tokenre, (3) Supabase SQL lekérdezés végrehajtása, (4) JSON válasz visszaküldése. Mindez tipikusan 50-200ms alatt megy végbe.

☐ Mit tanultam ebből?

- ☐ A webalkalmazások kliens-szerver modellre épülnek: a bövebb HTTP kéréseket küld a szervernek
- ☐ A JSON az API-k általános adatformátuma: ember által olvasható, könnyen feldolgozható
- ☐ Az eCetatean frontend (Cloudflare) és backend (Vercel) el van választva biztonsági és skálázási okokból
- ☐ A frontend fetch() hívással kommunikál a backenddel, JWT token-t küldve minden keresésben
- ☐ A backend Hono.js routeok feldolgozzák a kéréseket, lekérdezik az adatbázist, és JSON-ban válaszolnak

2. fejezet: TypeScript alapok

Az eCetatean projekt 100%-ban TypeScript-ben írt -- mind a frontend, mind a backend. Ez nem véletlen: a TypeScript a JavaScript egy bővitménye, amely statikus típusossá teszi a nyelvet. Ez azt jelenti, hogy a hibák egy része már fordításkor kiderül, nem csak futásidőben.

2.1 Miért TypeScript, nem JavaScript?

Kepzelj el egy függvényt, amely egy profil objektumot kap paraméterül. JavaScript-ben semmit sem tudunk a paraméterről -- lehet string, lehet null, lehet hiányozni is. TypeScript-ben viszont:

```
// JavaScript - nincs típusinformáció
function greetUser(profile) {
  return "Buna, " + profile.full_name // Mi van ha full_name null?
}

// TypeScript - a fordító ellenőrzi
function greetUser(profile: Profile): string {
  return "Buna, " + (profile.full_name ?? "cetateanean")
  // A fordító tudja, hogy full_name string | null
  // és figyelmezteti, ha nem kezelsz a null esetet
}
```

A TypeScript előnyei a projektben: Fordítási hiba jelzi ha rossz mezőnevet írsz (pl. profil.fullName helyett), Autocomplete az IDE-ben (pontosan tudja milyen mezők vannak), Öndetektáló kód (a típusdefiníciók dokumentációt is szolgálnak).

2.2 A TypDefiníciók a projektben

A backend/src/types/index.ts fájl tartalmazza az összes fontos típus definíciót. Nézzük meg a Profile interfációt:

```
// backend/src/types/index.ts
export interface Profile {
  id: string // UUID, sohasem null - minden profilnak van
  user_id: string // A Supabase Auth user azonosítója
  full_name: string | null // | null = opcionális mező, lehet hiányzó
  cnp: string | null // Roman személyi szám (13 számjegy)
  date_of_birth: string | null // ISO dátum: "1985-03-12"
  bulletin_series: string | null // Személyigazolvány sorozat: pl. "KX"
  bulletin_number: string | null // Személyigazolvány szám: pl. "123456"
  bulletin_expiry: string | null // Lejárati dátuma
  address: string | null // Lakcím
  city: string // Város, NOT NULL, default 'Cluj-Napoca'
  phone: string | null
  email: string | null
  language: 'ro' | 'hu' // Unió típus! Csak 'ro' vagy 'hu' lehet
  created_at: string // ISO timestamp
  updated_at: string
}
```

Figyeld meg a `language: 'ro' | 'hu'` sort -- ez egy unió típus. A fordító nem engedi, hogy más értéket adj meg. Ha megpróbálnál `language: 'en'` írni, fordítási hibát kapsz.

2.3 A Vehicle interface

```
export interface Vehicle {
  id: string
  user_id: string
  plate_number: string      // Kotelez! Minden jarmunek kell rendszam
  make: string | null       // Marka (pl. "Dacia")
  model: string | null      // Modell (pl. "Logan")
  year: number | null       // Szam tipusa! Nem string
  engine_cc: number | null  // Hengerurtartalom kubikcentiméterben
  fuel_type: string | null  // "benzin", "diesel", "electric"
  color: string | null
  vin: string | null        // Alvazszam (17 karakter)
  itp_expiry: string | null // ITP lejartat datuma
  rca_expiry: string | null // Kotelez biztositas lejarata
  impozit_amount: number | null // Adosszeg RON-ban
  created_at: string
}
```

Az `year: number | null` kulcsfontos: ha ezt `string | null`-nak definialnad, matematikai muveleteket nem lehetne rajta vegezni. A TypeScript tipus rendszer kenszeri a helyes adattipus hasznalatat.

2.4 AuditActionType - Union tipus

```
// backend/src/types/index.ts
export type AuditActionType =
  | 'login'
  | 'profile_updated'
  | 'vehicle_added'
  | 'vehicle_updated'
  | 'pdf_generated'
  | 'report_submitted'
  | 'chat_session'
  | 'life_event_started'
  | 'life_event_step_completed'
  | 'payment_handoff_started'
  | 'translation_document_completed'
  // ... es tovaabbi tipusok

// Hasznalat:
writeAuditEntry({
  userId,
  action: 'Profil frissitve',
  actionType: 'profile_updated', // Csak ezek a string ertekek ervenyek!
  data: { ... }
})
```

Ez a Pattern rendkivul ertekes: ha uj audit tip hozzaadsz, a fordito megmondja minden helyet ahol kezelni kell. Nem lehet elfelejteni.

2.5 Interface vs Type kulonbseg

Az eCetatean-ben mindkettot hasznaljak. A gyakorlati kulonbseg:

```
// Interface - bovittheto (extends, implements)
interface Profile { full_name: string | null }
interface ExtendedProfile extends Profile {
  avatar_url: string // + uj mezo
}

// Type alias - rugalmasabb, union/intersection lehetseges
type StepStatus = 'pending' | 'in_progress' | 'completed' | 'skipped'
type ApiResponse<T> = ApiSuccess<T> | ApiError // Generic union
```

2.6 Generikus tipusok a projektben

```
// backend/src/types/index.ts
export interface ApiSuccess<T> {
  success: true
  data: T // T = barmi lehet (Profile, Vehicle[], stb.)
}

export interface ApiError {
  success: false
  error: string
  code?: string // ? = opcionalis mezo
}

export type ApiResponse<T> = ApiSuccess<T> | ApiError

// Pelda hasznalat:
const profileResponse: ApiResponse<Profile> = {
  success: true,
  data: { full_name: "Kovacs Janos", ... }
}
// TypeScript tudja: ha success===true, akkor data Profile tipusu
```

2.7 Hogyan hasznaljak a tipusokat a route-okban?


```
// backend/src/routes/profile.ts
import type { Profile } from '../types'

profileRoute.post('/', requireAuth, async (c) => {
  let body: Partial<Profile> // Partial<T> = minden mezo opcionalis
  try {
    body = await c.req.json() // TypeScript tudja a tipust
  } catch {
    return c.json({ success: false, error: '...' }, 400)
  }

  const row = {
    user_id: userId,
    full_name: body.full_name ?? null,
    // body.full_name tipusa: string | null | undefined
    // ?? null: ha undefined, legyen null (adatbazis kompatibilis)
    city: body.city ?? 'Cluj-Napoca',
    // Ha nincs megadva varos, legyen Cluj-Napoca alapertelmezett
  }
})
```

A `Partial` TypeScript beépített utility típus: az eredeti interface összes mezőjét opcionálissá teszi. Ez tökéletesen illeszkedik ahhoz, hogy a felhasználó nem köteles minden mezőt kitölteni.

☐ Mit tanultam ebből?

- ☐ A TypeScript statikus típusosságot ad a JavaScript-hez: hibák fordítaskor derülnek ki
- ☐ Az interface-k leírják az objektumok alakját: Profile, Vehicle, AuditEntry stb.
- ☐ A `| null` azt jelenti: a mező lehet szöveg vagy null (opcionális adat)
- ☐ Az union típusok (`'ro' | 'hu'`) korlátozzák az érvényes értékeket
- ☐ A generikus típusok (ApiResponse) újrafelhasználható sablonak
- ☐ A Partial minden mezőt opcionálissá tesz - hasznos frissítéseknel

3. fejezet: React 19 es komponensek

A frontend React 19-et használ -- a Facebook által létrehozott UI könyvtár legújabb verzióját. A React alapötlete egyszerű: a UI függvényekből áll, amelyek adatot kapnak (props) és HTML-t adnak vissza (JSX). Ha az adat változik, a React automatikusan újrendereli az érintett részeket.

3.1 Mi az a React komponens?

```
// Egy egyszerű React komponens
function ProfileCard({ name, city }: { name: string; city: string }) {
  return (
    <div className="rounded-2xl border p-4">
      <h2 className="font-bold text-lg">{name}</h2>
      <p className="text-gray-500">{city}</p>
    </div>
  )
}

// Használat más komponensben:
<ProfileCard name="Kovacs Janos" city="Cluj-Napoca" />
```

A JSX (JavaScript XML) első ránézésre HTML-nek tűnik, de valójában JavaScript. A {} jelek között bármi JavaScript kifejezést beírhatasz. A `className` az oka miatt nem egyszerűen `class`: a `class` foglalt szó JavaScriptben.

3.2 A Protected komponens - Védetem utvonal

A frontend/src/lib/auth-guard.tsx tartalmazza a route védelmére használt komponenst, amely pontosan mutatja a React működését:

```
// frontend/src/lib/auth-guard.tsx
export function Protected({ children }: { children: ReactNode }) {
  const { isLoading, isSignedIn } = useAuth()
  // useAuth() = custom hook, visszaadja a bejelentkezési állapotot

  if (!isLoading) {
    // Meg töltődik - mutassunk egy loading UI-t
    return (
      <div className="flex min-h-dvh items-center justify-center bg-bg">
        <div className="anim-typing flex gap-1.5 items-center">
          <span className="w-2 h-2 rounded-full bg-text-tertiary" />
          <span className="w-2 h-2 rounded-full bg-text-tertiary" />
          <span className="w-2 h-2 rounded-full bg-text-tertiary" />
        </div>
      </div>
    )
  }

  if (!isSignedIn) {
    return <Navigate to="/auth" replace />
    // Nem bejelentkezett? Átirányítás a bejelentkezési oldalra
  }

  return <>{children}</>
  // Bejelentkezett! Megjeleníti a gyerek komponenseket
}
```

3.3 useState: lokális állapot

A `useState` hook lehetővé teszi, hogy egy komponens 'emlekezzen' valamire újrenderelés között. Az auth.tsx fájlban látni ezt:

```
// frontend/src/routes/auth.tsx
function Auth() {
  const [tab, setTab] = useState<"signin" | "signup">("signin")
  // tab: aktualis ertek, "signin" az alapertelmezett
  // setTab: fuggveny az ertek megvaltoztatasa

  const [email, setEmail] = useState("")
  const [password, setPassword] = useState("")
  const [error, setError] = useState<string | null>(null)
  const [loading, setLoading] = useState(false)

  // Amikor setTab("signup") hivodik, React ujrendereli
  // a komponenst, es a tab erteke "signup" lesz

  return (
    <div>
      <button onClick={() => setTab("signin")}>Bejelentkezés</button>
      <button onClick={() => setTab("signup")}>Regisztráció</button>
      { /* Feltetes renderelés: */ }
      {tab === "signin" ? <SignInForm /> : <SignUpForm />}
    </div>
  )
}
```

3.4 useEffect: mellekhatások kezelése

A `useEffect` hook lehetove teszi, hogy a komponens reagáljon változásokra és mellekhatásokat hajtson végre (API hívások, DOM manipuláció, timers stb.). A chat.tsx fájlban:

```
// frontend/src/routes/chat.tsx
useEffect(() => {
  if (profileLoading) return // Még töltődik, várjunk

  if (conversationId) {
    if (convLoading) return
    if (hydratedRef.current === conversationId) return
    if (convData) {
      // Betöltött beszélgetés üzeneteit beállítja
      const mapped = storedMessagesToMsgs(convData.messages)
      setMsgs(mapped.length > 0 ? mapped : [buildGreeting(displayName)])
      hydratedRef.current = conversationId
    }
    return
  }

  // Új beszélgetés: udvozlo üzenet
  hydratedRef.current = null
  setMsgs((prev) => {
    if (prev.length > 1) return prev
    return [buildGreeting(displayName)]
  })
}, [conversationId, convData, convLoading, displayName, profileLoading])
// A tombbeli ertekek a 'fuggosegek': ha barmelyik valtozik,
// az useEffect ujra lefut
```

3.5 Custom hook: useProfile

A custom hook-ok egyszeruen fuggvények, amelyek mas hook-okat használnak. A frontend/src/lib/api-hooks.ts fajlban:

```
// frontend/src/lib/api-hooks.ts
export function useProfile() {
  const getToken = useGetToken()
  const { isSignedIn, userId } = useAuth()

  return useQuery({
    // TanStack Query: adatletoltes + cacheles
    queryKey: ["profile", userId],
    // Egyedi azonosito: ha mar letoltottuk, cache-bol jon vissza
    queryFn: () => apiGet<CitizenProfile | null>("/api/profile", getToken),
    // Csak ha be van jelentkezve futtassa a kerdest
    enabled: !!isSignedIn && !!userId,
  })
}

// Hasznalat barmely komponensben:
function Home() {
  const { data: profile, isLoading } = useProfile()
  // data = CitizenProfile | null
  // isLoading = true amig toltodik
  if (isLoading) return <Spinner />
  return <div>{profile?.full_name}</div>
}
```

A TanStack Query automatikusan cacheli az adatot, ujra letolti ha szukseges, es kezeli a betoltesi es hibaallapotokat. Nini, nincs szukseg useEffect + useState kombinaciora! Ez a modern React fejlesztés elonye.

3.6 TanStack Router: fajl-alapu routing

A projekt TanStack Router-t hasznal, ahol az utvonalak fajlok. A frontend/src/routes/ mappaban minden fajl egy URL-nek felel meg:

```
// A fajl neve határozza meg az URL-t:
// routes/home.tsx      => /home
// routes/chat.tsx      => /chat
// routes/auth.tsx      => /auth
// routes/profile.tsx   => /profile
// routes/documents.tsx => /documents

// A route definicioja a fajlban:
// frontend/src/routes/chat.tsx
export const Route = createFileRoute("/chat")({
  head: () => ({ meta: [{ title: "Claudia -- eCetatean" }] }),
  component: () => (
    <Protected>
      <Chat />
    </Protected>
  ),
})
```

A `Protected` wrapper ellenorzi, hogy be van-e jelentkezve a felhasznalo. Ha nem, automatikusan atiranyitja a /auth oldalra.

☐ **Mit tanultam ebbol?**

- ☐ A React komponens egy fuggveny amely JSX-et ad vissza (mint HTML, de valojaban JavaScript)
- ☐ A Props lehetove teszi az adatok lefele aramlasat a szulo komponstol a gyerekig
- ☐ A useState az adatok 'emlekezete': ha valtozik, a komponens ujrenderelodik
- ☐ A useEffect mellekhatásokra reagál: API hívások, betoltes, stb.
- ☐ A custom hook-ok (useProfile, useNews) ujrafelhasznalhato adatlogikat csoportositanak
- ☐ A TanStack Router fajl-alapu routingot nyujt: minden fajl = egy URL

4. fejezet: UI reteg: Tailwind CSS es shadcn/ui

Az eCetatean UI realistikus, modern kinezetu mobilalkalmazast utanoz, tele animaciokkal es aternyezo dizajnelemekkel. Ennek ket fo eszkoze a Tailwind CSS es a shadcn/ui konyvtar.

4.1 Tailwind CSS: utility-first megkozelites

A hagyomanyos CSS-ben kulsz fajlokat irsz es osztaly neveket adasz az elemeknek. A Tailwind mas utat jart: minden stilus egy kis utility osztaly. Ehelyett, hogy `.card { border-radius: 16px; padding: 16px; }` irsz, az osztalyokat kozvetlenul az elemre rakod:

```
// Tailwind osztalyok a home.tsx-bol:
<button
  onClick={() => nav({ to: a.to as "/chat" })}
  className="press flex items-center gap-3 px-4 py-3.5 rounded-2xl
            border border-border bg-surface shadow-card
            hover:shadow-elevated transition-shadow text-left"
>

// Osztaly magyarázat:
// press      = custom: lenyomashatosag vizualis visszajelzese
// flex       = display: flex (flexbox)
// items-center = align-items: center (vertikalis kozepre)
// gap-3      = rem egysegekben koz a flex gyermekek kozott
// px-4       = padding balra-jobbra 1rem
// py-3.5     = padding fel-le 0.875rem
// rounded-2xl = border-radius 1rem (16px)
// border     = border: 1px solid
// border-border = custom szin a border-ra (CSS variable)
// bg-surface  = hattar szin (CSS variable, vilagos/sotet tema)
// shadow-card = arnyekozas (custom CSS variable)
// hover:shadow-elevated = hover allapotban nagyobb arnyekozas
// transition-shadow = animalt atvaltas
```

A Tailwind fobb elonyei: nem kell nevet kitalalni CSS osztalyoknak, nem no a CSS fajl merete (csak a hasznalt osztalyok kerulnek be a bundle-be), es a stilus kozvetlenul latszik a JSX-ben -- nincs konstans fajlvaltozas.

4.2 Tobbszoros Tailwind osztaly kombinacio

A home.tsx FeatureGrid komponenseben latszik a bonyolultabb hasznalat:

```
// frontend/src/routes/home.tsx - FeatureGrid
<motion.button
  key={card.id}
  onClick={() => onCardClick(card.id)}
  className={cn(
    "press rounded-[20px] bg-gradient-to-br text-left flex flex-col",
    "border border-black/[0.07] shadow-card overflow-hidden",
    card.bgClass, // Dinamikus: "from-[#0E7C66]/10 to-[#0E7C66]/4"
  )}
  style={{ aspectRatio: "196 / 230", color: card.accentColor }}
  aria-label={`Deschide ${card.title}`}
>

// cn() = class merging utility (clsx + tailwind-merge)
// Ha ellentmondó osztályok vannak, az utolsó nyer
// rounded-[20px] = tetszőleges érték szögletes zárójellel
```

4.3 shadcn/ui: Radix UI alapu komponensek

A shadcn/ui nem egy hagyományos könyvtár -- nem telepítod npm-mel, hanem a komponensek forráskódját másolod a projektbe. Ezek a Radix UI primitiveire épülnek, amelyek accessibility-t (ARIA attribútumok, keyboard navigáció) biztosítanak. A home.tsx-ban látható Dialog komponens:

```
// frontend/src/routes/home.tsx - QuickActionSheet
import * as Dialog from "@radix-ui/react-dialog"

function QuickActionSheet({ cardId, onClose }) {
  return (
    <Dialog.Root open={!cardId} onOpenChange={(open) => !open && onClose()}>
      <Dialog.Portal>
        {/* Overlay: sötét háttér */}
        <Dialog.Overlay className="fixed inset-0 z-40 bg-black/25 backdrop-blur-[2px]" />
        {/* Content: a panel maga */}
        <Dialog.Content asChild>
          <motion.div
            initial="hidden"
            animate="visible"
            exit="hidden"
            variants={slideUpSheet} // Framer Motion animáció
            className="fixed bottom-0 left-0 right-0 z-50
              bg-surface rounded-t-[22px] shadow-sheet"
          >
            <Dialog.Close className="press ml-auto w-8 h-8 rounded-full ..." >
              <X size={15} />
            </Dialog.Close>
          </motion.div>
        </Dialog.Content>
      </Dialog.Portal>
    </Dialog.Root>
  )
}
```

A Radix Dialog automatikusan: (1) focus trapping (fókusz a dialogon belül marad), (2) Escape gombon becsukodik, (3) ARIA attribútumok (role, aria-modal), (4) scroll lock. Mindezt elerjük

anelkul, hogy magunk megirnenk.

4.4 Framer Motion animaciok

A projekt Framer Motion-t hasznal animaciokhoz. A motion elemek css transition helyett JavaScript-alapu animaciot hasznalnak, amely komplexebb effektusokat tesz lehetove:

```
// frontend/src/lib/motion.ts (valoszinusitheto tartalma)
export const slideUpSheet = {
  hidden: { y: "100%", opacity: 0 },      // Kiindulo allapot
  visible: {
    y: 0, opacity: 1,                      // Vegso allapot
    transition: { type: "spring",          // Rugoszeru animacio
      damping: 25, stiffness: 300 }
  }
}

// Hasznalat:
<motion.div
  initial="hidden"      // Mikor megjelenik: hidden-bol indul
  animate="visible"     // Animate to visible allapot
  exit="hidden"         // Mikor eltavozik: visszamegy hidden-be
  variants={slideUpSheet} // A fenti animacio definicio
/>
```

☐ Mit tanultam ebbol?

- ☐ A Tailwind CSS utility osztályokat hasznal CSS fajok helyett: minden stilus az elemnel van
- ☐ A cn() fuggveny okosan egyesiti a Tailwind osztalyokat (clsx + tailwind-merge)
- ☐ A shadcn/ui accessibility-t kapunk ingyen: keyboard navigacio, ARIA, focus management
- ☐ A Radix UI primitives az accessibility ret alapja (Dialog, Dropdown, Tooltip stb.)
- ☐ A Framer Motion spring-alapu animaciokat tesz lehetove deklarativ modon

5. fejezet: Backend: Hono.js es REST API-k

A backend egy HTTP szerver, amely 3001-es porton hallgat (fejlesztésben), vagy Vercel serverless functionként fut (produkcióban). A Hono.js egy ultrakönnyű TypeScript web framework, amely kifejezetten edge environment-ekre (Cloudflare Workers, Vercel Edge) terveztek.

5.1 Mi a web szerver?

Egy web szerver egy program, amely hallgat egy portot, fogadja a HTTP kéréseket, és válaszol rájuk. Az eCetatean backend Node.js-ben fut, de Vercel-en serverless funkcióként deployol. Minden egyes API kérésem egy új 'szalat' indít el.

5.2 A Hono.js app setup (backend/src/index.ts)

```

// backend/src/index.ts - A teljes szerver beleptesi pontja
import { Hono } from 'hono'
import { logger } from 'hono/logger'
import { corsMiddleware } from './middleware/cors'
import { claudiaRoute } from './routes/claudia'
import { profileRoute } from './routes/profile'
// ... további route import-ok

const app = new Hono()

// ---- Globalis Middleware -----
app.use('*', logger()) // Minden kereset naploz (URL, ido, statusz)
app.use('*', corsMiddleware) // CORS headerek hozzaadasa

// ---- Route-ok regisztralasa -----
app.route('/api/claudia', claudiaRoute) // AI chat
app.route('/api/profile', profileRoute) // Profil CRUD
app.route('/api/vehicles', vehiclesRoute) // Jarmu kezeles
app.route('/api/pdf', pdfRoute) // PDF generalas
app.route('/api/news', newsRoute) // Hirek
// ... további route-ok

// ---- 404 Handler -----
app.notFound((c) => c.json({ success: false, error: 'Endpoint negasit' }, 404))

// ---- Hibakezeles -----
app.onError((err, c) => {
  console.error('Unhandled error:', err)
  return c.json({ success: false, error: 'Eroare interna' }, 500)
})

// ---- Helyi fejlesztő szerver indítás -----
if (process.env.VERCEL !== '1') {
  const port = Number(process.env.PORT ?? 3001)
  import('@hono/node-server').then(({ serve }) => {
    serve({ fetch: app.fetch, port })
    console.log(`eCetatean API listening on http://localhost:${port}`)
  })
}

export default app // Vercel ezt a modult importálja

```

A `process.env.VERCEL !== '1'` ellenorzes kulcsfontossagu: Vercel-en NEM kell portot nyitni, mert a platform maga kezeli a HTTP hallgatast. Az `export default app` sorral Vercel azt kapja, amire szuksege van: a Hono app `fetch` metodusat.

5.3 REST API konvencioik

GET <code>/api/vehicles</code>	Osszes jarmU listazasa (olvasas)
POST <code>/api/vehicles</code>	Uj jarmu létrehozasa (body-ban az adatok)
PATCH <code>/api/vehicles/:id</code>	Jarmu frissitese (reszleges)

DELETE /api/vehicles/:id	Jarmu torlese
GET /api/profile	Sajat profil lekerese
POST /api/profile	Profil mentese/frissitese

Az `:id` egy URL parameter -- a Hono `c.req.param('id')`-vel eri el. Peldaul: `PATCH /api/vehicles/abc-123-def` eseten az `id` erteke `abc-123-def`.

5.4 Jarmu endpoint -- teljes peldaval

```
// backend/src/routes/vehicles.ts
vehiclesRoute.get('/', requireAuth, async (c) => {
  const userId = c.get('userId')
  // userId-t a requireAuth middleware allitotta be (JWT alapjan)

  if (!isSupabaseConfigured) {
    return c.json({ success: true, data: [] })
    // Ha nincs adatbazis, ures tombot adunk vissza
  }

  const { data, error } = await supabaseAdmin
    .from('vehicles')
    .select('id, plate_number, make, model, year, vin, ...')
    .eq('user_id', userId) // WHERE user_id = ?
    .order('created_at', { ascending: false }) // ORDER BY created_at DESC

  if (error) {
    return c.json({ success: false, error: 'Hiba...' }, 500)
  }

  return c.json({ success: true, data: data ?? [] })
  // 200 OK + JSON valasz
})
```

5.5 Az auth middleware (middleware/auth.ts)

A `requireAuth` middleware minden vedett endpoint elott fut. Ellenorzi a JWT tokent es beallitozza a userId-t:

```
// backend/src/middleware/auth.ts
export const requireAuth = createMiddleware(async (c, next) => {

  // ---- Fejlesztési gyorsbillentu -----
  // AUTH_STUB=true eseten atugorjuk a JWT ellenorzest
  // SOHASEM hasznald produkcioban!
  if (process.env.AUTH_STUB === 'true') {
    c.set('userId', 'stub-user-citizen')
    c.set('userRole', 'citizen')
    return next()
  }

  // ---- Valos JWT ellenorzes -----
  const authHeader = c.req.header('Authorization')

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return c.json({ success: false, error: 'Token hianyzik' }, 401)
  }

  const token = authHeader.replace('Bearer ', '')
  // Az 'Authorization: Bearer eyJhbGci...' headerbol kivonjuk a tokent

  try {
    const payload = await verifySupabaseJWT(token)
    c.set('userId', payload.sub) // User azonosito beallitozasa
    c.set('userRole', payload.role) // Szerep beallitozasa
    c.set('clerkPayload', payload) // Teljes payload mentese
    await next() // Kovetkezo handler futtatasa
  } catch (err) {
    return c.json({ success: false, error: 'Token ervenytelen' }, 401)
  }
})
```

Az `AUTH\_STUB=true` környezeti változó fejlesztőknek hasznos: nem kell bejelentkezni a fejlesztés során. A `.env.local` fájlban beállítjuk, és a backend atugorbja a JWT ellenorzest.

❑ Mit tanultam ebből?

- ❑ A Hono.js egyszerű, típusbiztos web framework Edge és Node.js környezetekhez
- ❑ A route-ok hierarchikusan szervezett HTTP endpointok (GET/POST/PATCH/DELETE)
- ❑ A middleware köztes szoftver: logger, CORS, auth -- minden keresnel lefut
- ❑ A requireAuth middleware ellenorzi a JWT tokent és beállítja a userId-t a kontextusba
- ❑ Az AUTH_STUB=true fejlesztési gyorsbillentu: atugorja a JWT ellenorzest
- ❑ A Vercel-en az `export default app` elegendő -- nem kell portot nyitni

6. fejezet: Adatbázis: Supabase es PostgreSQL

Az eCetatean adatait egy PostgreSQL adatbázisban tarolja, amelyet a Supabase platform kezel. A Supabase nyújt: (1) Hosztolt PostgreSQL adatbázist, (2) REST API-t (PostgREST), (3) Autentikációs szerveret, (4) Fajltárolót (Storage), (5) Valós idejű subscriptionokat. Ingyenes tier is elérhető.

6.1 Mi a relacionális adatbázis?

Kepzelj el egy Excel táblát. Az adatbázis hasonló: táblákból áll, amelyek sorokból és oszlopokból állnak. A különbség: az adatbázis kényszeríthető szabályokat (NOT NULL, UNIQUE, FOREIGN KEY), jobb teljesítményt nyújt (indexek), és biztos az adatkonzisztencia (tranzakciók).

Az eCetatean adatbázisa táblák: profiles, vehicles, civic_reports, audit_log, pdf_forms, life_event_progress, news, chat_conversations, chat_messages.

6.2 A profiles tábla schema.sql-ből

```
-- backend/schema.sql
create table if not exists public.profiles (
  id                uuid primary key default gen_random_uuid(),
  -- uuid = universally unique identifier, gen_random_uuid() auto-general
  user_id           text not null unique,
  -- text = szöveg, not null = nem lehet üres, unique = minden usernek 1 sor
  full_name         text,                    -- Nullable: lehet üres
  cnp                text,                  -- Roman személyi szám
  date_of_birth     date,                   -- Datum típus
  bulletin_series   text,
  bulletin_number   text,
  bulletin_expiry   date,
  address           text,
  city              text not null default 'Cluj-Napoca',
  -- default értéket ad ha nem adjuk meg
  phone             text,
  email             text,
  language          text not null default 'ro' check (language in ('ro', 'hu')),
  -- check: csak 'ro' vagy 'hu' lehet, mint a TypeScript union típus!
  created_at        timestampz not null default now(),
  updated_at        timestampz not null default now()
);
-- Indexek: gyors keresést tesznek lehetővé
create index if not exists profiles_user_id_idx on public.profiles (user_id);
create index if not exists profiles_cnp_idx on public.profiles (cnp);
```

6.3 A vehicles tábla

```

create table if not exists public.vehicles (
  id                uuid primary key default gen_random_uuid(),
  user_id           text not null,
  -- FOREIGN KEY kapcsolat: profiles.user_id-re mutat (implicitten)
  plate_number      text not null, -- Kotelezo rendszam
  make              text,          -- Marka: "Dacia", "BMW" stb.
  model             text,
  year              int,           -- Egesz szam
  engine_cc         int,          -- Hengerurtartalom
  fuel_type         text,
  color             text,
  vin               text,          -- 17-karakteres alvazszam
  itp_expiry        date,          -- ITP lejarat datuma
  rca_expiry        date,          -- Biztositas lejarat datuma
  impozit_amount    numeric,      -- Decimalis: 234.50 RON
  created_at        timestamptz not null default now()
);
create index if not exists vehicles_user_id_idx on public.vehicles (user_id);
-- Index nelkul: O(n) keresni 1 millio jarmu kozott
-- Indexszel: O(log n) - sorrendnyi gyorsabb!

```

6.4 Miert fontosak az indexek?

Ha egy tablanel nincs index es 1 millio sor van benne, az adatbazisnak minden sort at kell nezni, hogy megtalalala a keresett profilt. Az index olyan, mint egy konyv targymutatoja: kozvetlenul a megfelelo lapra ugrik.

A `profiles\_user\_id\_idx` index azert fontos, mert MINDEN API keresnel `WHERE user\_id = ?` feltetelje van. Index nelkul ez nagyon lassu lenne.

6.5 Row Level Security (RLS)

Az eCetatean projektben az RLS KI VAN KAPCSOLVA, mert a backend service_role kulcssal csatlakozik (kivesezetett), amely alapbol atugorja az RLS-t. A schema.sql megjegyzi: 'add production policies before enabling it'. Ha RLS be lenne kapcsolva, ez lenne egy tipikus policy:

```

-- Ha RLS be lenne kapcsolva (produkcioban):
alter table public.profiles enable row level security;

create policy "Users can only see their own profile"
  on public.profiles
  for select
  using (auth.uid()::text = user_id);
-- auth.uid() = a bejelentkezett felhasznalo azonositoja
-- Mindenki csak a SAJAT profiljat lathatja!

create policy "Users can update their own profile"
  on public.profiles
  for update
  using (auth.uid()::text = user_id)
  with check (auth.uid()::text = user_id);

```

6.6 A Supabase kliens inicializalasa

```
// backend/src/lib/supabase.ts
import { createClient } from '@supabase/supabase-js'

const url = process.env.SUPABASE_URL // Supabase projekt URL
const key = process.env.SUPABASE_SERVICE_ROLE_KEY // Titkos kulcs!

export const isSupabaseConfigured = Boolean(url && key)

export const supabaseAdmin = createClient(
  url || 'http://localhost:54321',
  key || 'placeholder-service-role-key',
  {
    auth: {
      autoRefreshToken: false, // Backend nem frissíti a token-t
      persistSession: false, // Nem tárolja a munkamenetet
    },
  }
)

// FONTOS: a service_role kulcs átadja az RLS-t
// Sohasem küld ki a frontendre!
```

6.7 Valos Supabase lekérdezés

```
// Profil mentése (backend/src/routes/profile.ts)
const row = {
  user_id: userId,
  full_name: body.full_name ?? null,
  city: body.city ?? 'Cluj-Napoca',
  // ...
}

const { data, error } = await supabaseAdmin
  .from('profiles') // FROM profiles
  .upsert(row, { onConflict: 'user_id' })
  // INSERT ... ON CONFLICT (user_id) DO UPDATE SET ...
  // Ha már van profil az adott user_id-vel, frissíti!
  .select('full_name, cnp, email, ...')
  .single() // Pontosán 1 sort várunk vissza

if (error) {
  console.error('profile upsert error:', error.message)
  return c.json({ success: false, error: 'Hiba' }, 500)
}
return c.json({ success: true, data })
```


☐ **Mit tanultam ebbol?**

- ☐ A PostgreSQL relationalas adatbazis: tablak, sorok, oszlopok, kulcsok
- ☐ Az UUID primary key auto-general egyedi azonositot minden sorhoz
- ☐ Az indexek drantikusan gyorsitjak a keresest: $O(n)$ helyett $O(\log n)$
- ☐ A Supabase service_role kulcs atugorja az RLS-t -- csak backenden hasznald
- ☐ Az upsert = INSERT OR UPDATE: ha letezik, frissit; ha nem, beilleszt
- ☐ Az isSupabaseConfigured flag lehetove teszi mock adatokat ha nincs adatbazis

7. fejezet: Autentikacio

Az autentikacio valaszolja meg: 'Ki vagy te?' A Supabase Auth JWT-t (JSON Web Token) hasznal a felhasznalok azonositasara. Minden bejelentkezes utan a Supabase kiad egy JWT tokent, amelyet a frontend elment es minden API keresnel elkuldoz.

7.1 Session vs JWT alapu autentikacio

Session-alapu	A szerver tartja nyilvan ki van bejelentkezve (sessionStore). Allapotos.
JWT-alapu	A token maga tartalmazza az adatokat. A szerver csak ellenorzi az alairasat. Allapotmentes.
Supabase Auth	JWT-t hasznal. Bejelentkezes utan access_token + refresh_token kerul a kliensre.

7.2 A JWT struktura

A JWT harom, ponttal elvalasztott reszbol all: header.payload.signature. Mindenki meg tudja nezni a tartalmat (Base64 kodolt), de megvaltoztatni nem -- a signature ellenorzi az integritast.

```
// JWT token peldaja (rorviditve):
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9  <- Header (algoritmus)
.eyJzdWIiOiJ1c2VyLTExMyIsImVtYWlsIjoiam9obi4uLi4iLCJyb2xl... <- Payload
.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c  <- Signature

// Dekodolt payload:
{
  "sub": "user-abc-123",           // User azonosito (subject)
  "email": "kovacs@example.com",
  "role": "authenticated",
  "user_metadata": { "role": "citizen" },
  "iat": 1748380800,              // Kiadasi ido (Unix timestamp)
  "exp": 1748384400,              // Lejarati ido (1 ora mulva)
  "aud": "authenticated"         // Kozonseg (audience)
}
```

7.3 JWT ellenorzes (supabase-jwt.ts)

```
// backend/src/lib/supabase-jwt.ts
import { jwtVerify } from 'jose'

export async function verifySupabaseJWT(token: string): Promise<ClerkPayload> {
  const secret = new TextEncoder().encode(process.env.SUPABASE_JWT_SECRET!)
  // A titkos kulcs: csak a szerver ismeri
  // .encode() = string-ból Uint8Array (bytes) konverzio

  try {
    const { payload } = await jwtVerify(token, secret, {
      audience: 'authenticated' // Ellenorzi az 'aud' mezot
    })
    // A jwtVerify:
    // 1. Ellenorzi a signature-t (titkos kulccsal)
    // 2. Ellenorzi a lejarsasi idot (exp)
    // 3. Ellenorzi az audience-t (aud)
    // Ha barmely ellenorzes megbukik: exception!

    const role = appMeta?.app_role ?? 'citizen'
    return {
      sub: payload.sub!, // User azonosito
      email: payload.email,
      role, // citizen | civil_servant | admin
      exp: payload.exp ?? 0,
      iat: payload.iat ?? 0,
    }
  } catch {
    throw new Error('Token Supabase invalid')
  }
}
```

7.4 Bejelentkezési folyamat a frontenden

```
// frontend/src/routes/auth.tsx
async function handleSignIn(e: React.FormEvent) {
  e.preventDefault()
  setLoading(true)

  const { error: authError } = await supabase.auth.signInWithPassword({
    email, password
  })
  // Supabase kliens elkuldja a bejelentkezési adatokat
  // Supabase visszakuld: { session: { access_token, refresh_token }, user }

  setLoading(false)
  if (authError) {
    setError("Email vagy jelszo helytelen.")
    return
  }
  // Ha sikeres: Supabase Auth kliens automatikusan elmenti a tokent
  // A useAuth() hook ezutan isSignedIn = true-t ad vissza
}
```

7.5 EidKit OIDC: elektronikus személyazonosság ellenorzes

Az eCetatean támogatja az EidKit-et, amely az OIDC (OpenID Connect) protokollt használja. Az OIDC az OAuth 2.0-ra épül, kiegészítődik azonosság adatokkal. Az authorization code flow menete:

- Felhasználó kattint 'Azonosság EidKit-tel'
- A backend létrehoz egy 'state' és 'nonce' értéket (CSRF és replay védelem)
- Átirányít az EidKit login oldalra a state-tel
- Felhasználó azonosítja magát az EidKit-en (személyazonosság igazolványával)
- EidKit visszairányít a backend callback URL-jére az authorization code-dal
- A backend a code-ot tokenre cseréli, ellenőrzi a nonce-t
- A verifikálás eredménye elmentődik a profilba (identity_verified_at stb.)

☐ Mit tanultam ebből?

- ☐ A JWT (JSON Web Token) három részes token: header.payload.signature
- ☐ A payload tartalmazza a user adatokat; csak a signature ellenőrzése szükséges
- ☐ A jwtVerify ellenőrzi az aláírást, lejárati időt és az audience-t
- ☐ Az AUTH_STUB=true átugorja az ellenőrzést fejlesztésben
- ☐ Az OIDC (EidKit) authorization code flow-val verifikálja a személyazonosságot
- ☐ A Supabase Auth kezeli a session-t, token frissítést és kijelentkezést

8. fejezet: AI integracios

A ClaudeAI az eCetatean AI asszisztense -- egy nagy nyelvi modell (LLM) API-n keresztül kommunikál. A projekt hierarchiat alkalmaz: eloszor megprobalalja Anthropic Claude-dal, majd Google Gemini-vel, vegul egy mock valasszal. Ezt 'graceful degradation'-nek hivjak.

8.1 Mi az az LLM API?

Egy LLM (Large Language Model) API-nal uzenetek tombjet kuldjuk el (messages array), es a modell szoveget valaszol. Nincs allapot: minden kerelem teljes kontextust igyenel. A modellek 'tokenekkel' dolgoznak (kb. 4 karakter = 1 token), es token-limi van.

8.2 A Claude API hivas (claude-claudia.ts)

```
// backend/src/lib/claude-claudia.ts
const MODEL = 'claude-sonnet-4-6'

export async function streamClaudeClaudia(
  messages: ChatMessage[],
  profile: Partial<Profile> | undefined,
  enqueue: (line: object) => void
): Promise<void> {
  const client = new Anthropic({ apiKey })
  const lastUser = messages.filter(m => m.role === 'user').pop()?.content ?? ''
  const history = toAnthropicHistory(messages)

  const stream = client.messages.stream({
    model: MODEL, // claude-sonnet-4-6
    max_tokens: 1024, // Max 1024 token a valaszban
    system: buildSystemPrompt(profile), // Rendszer prompt
    tools, // Eszk. amit a modell hivhat
    messages: [
      ...history, // Elozo uzenetek (kontextus)
      { role: 'user', content: lastUser } // Legujabb uzenet
    ],
  })

  // Streaming: tokenenk erkeznek, azonnal kuld tovabb
  for await (const event of stream) {
    if (event.type === 'content_block_delta' &&
      event.delta.type === 'text_delta') {
      enqueue({ type: 'text', content: event.delta.text })
      // Minden token azonnal elkuldodik a frontendre!
    }
  }
  // ... (folytatás a fajlban)
}
```

8.3 A rendszer prompt

A system prompt határozza meg a modell viselkedését. A claude-claudia.ts-ben:

```
function buildSystemPrompt(profile?: Partial<Profile>): string {
  const eventsSummary = Object.values(LIFE_EVENTS)
    .map(e => `- ${e.event_type}: ${e.title} -- ${e.summary}`)
    .join('\n')
  // Az osszes eletese meny bekerul a promptba

  const profileBlock = profile?.full_name
    ? `\nProfil cetatean: ${profile.full_name}, CNP ${profile.cnp ?? '--}'`
    : ''

  return `Esti ClaudIA, asistent civic digital...
REGULI CRITICE:
- Raspunde INTOTDEAUNA in limba romana.
- Fii SCURT si DIRECT. Maxim 2-3 propozitii.
- NU folosi formatare markdown: fara **, fara #
- Pentru ORICE intrebare administrativa, foloseste un instrument.
${profileBlock}
Evenimente disponibile:
${eventsSummary}`
}
```

8.4 Tool use / Function calling

A modell nem csak szoveget valaszolhat -- 'eszkozokat' is hivhat. Az eszkozok definialva vannak a `claudia-claudia.ts`-ban:

```
const tools: Anthropic.Tool[] = [
  {
    name: 'handle_life_event',
    description: 'Genereaza plan de actiune pentru un eveniment civic',
    input_schema: {
      type: 'object',
      properties: {
        event_type: {
          type: 'string',
          description: 'Cheia evenimentului din baza de cunostinte'
        }
      },
      required: ['event_type']
    }
  },
  {
    name: 'find_procedure',
    description: 'Cauta o procedura administrativa in catalogul tipizatul.eu',
    input_schema: { ... }
  },
  // ... tova bbi eszkozok
]
```

Amikor a Claude ugy donto, hogy hasznalnia kell az 'handle_life_event' eszkozi, visszakuldi az eszkoz nevit es az inputot. A backend ezutan meghivja a `handleToolCall()` fuggvenyt a `claudia-tools.ts`-bol:

```
// backend/src/lib/claudia-tools.ts
export async function handleToolCall(
  toolName: string,
  input: Record<string, string>
) {
  switch (toolName) {
    case 'handle_life_event': {
      const procedure = findProcedure(input.event_type)
      const upgraded = await resolveProcedureActionPlanFormSlugs(procedure)
      return {
        type: 'action_plan' as const,
        procedure: upgraded,
        create_life_event: DEMO_LIFE_EVENT_TYPES.has(input.event_type),
        event_type: input.event_type,
      }
    }
    case 'find_procedure': {
      const matches = await findProcedures(input.query, { county: 'Cluj', limit: 5 })
      const plan = await synthesizeActionPlan(matches[0])
      return { type: 'tipizatul_action_plan', plan, alternatives: ... }
    }
    // ...
  }
}
```

8.5 Graceful degradation: Claude -> Gemini -> Mock

```
// backend/src/routes/claudia.ts
const useClaude = isClaudeConfigured()
const useGemini = !useClaude && isGeminiConfigured()

if (useClaude) {
  await streamClaudeClaudia(messages, profile, enqueue)
} else if (useGemini) {
  await streamGeminiClaudia(messages, profile, enqueue)
} else {
  // Nincs API kulcs - mock valasz
  const mock = getMockResponse(lastUserMessage)
  enqueue({ type: 'text', content: mock.text })
}
```

A Gemini implementacioban a MODEL_CANDIDATES tomb mutatja a 'model fallback chain-t': eloszor gemini-2.0-flash-lite (ingyenes), azutan gemini-2.0-flash, vegul gemini-2.5-flash. Ha kvota hiba jon, a GeminiQuotaError el kap es nem try-olja a kovetkező modellet.

8.6 Streaming a frontenden

```
// A Claude chat válasz streaming feldolgozása (chat.tsx)
// Az apiStreamPost egy Response objektumot ad vissza (nem JSON-t!)

const chunks: ClaudeStreamChunk[] = []
const reader = response.body?.getReader()
const decoder = new TextDecoder()

while (true) {
  const { done, value } = await reader.read()
  if (done) break
  const text = decoder.decode(value)
  // Minden sor egy JSON objektum:
  // {"type":"text","content":"Am"}
  // {"type":"text","content":" inteles"}
  // {"type":"tool_result","tool_name":"handle_life_event","result":{...}}
  for (const line of text.split('\n')) {
    if (line.trim()) chunks.push(JSON.parse(line))
  }
}
}
```

☐ Mit tanultam ebből?

- ☐ Az LLM API-k üzeneteket fogadnak és szöveget válaszolnak (token-onként)
- ☐ A system prompt határozza meg a modell viselkedését és kontextust ad
- ☐ A tool use / function calling lehetővé teszi, hogy az AI függvényeket hívjon
- ☐ A streaming azonnal küldözi a token-eket: a felhasználó nem vár a teljes válaszig
- ☐ A graceful degradation: Claude -> Gemini -> Mock -- mindig van válasz
- ☐ A GeminiQuotaError speciális kezelése: kvóta túllépés esetén nem próbálkozik tovább

9. fejezet: PDF generalas

Az eCetatean egyik fő funkciója: román közigazgatási űrlapok automatikus kitöltése a felhasználó profil adataival. Ez egy bonyolult probléma: a PDF-eknek nincsenek egységes mezőnevek, és az űrlapok különböznek.

9.1 @pdfme/pdf-lib vs pdf-lib

@pdfme/pdf-lib	A pdfme csomagban lévő fork. Az eCetatean ezt használja a PDF-ek betöltéséhez, AcroForm mezők kinyeréséhez és kitöltéséhez.
pdf-lib	Általános PDF manipulációs könyvtár: új PDF-ek létrehozása, szöveg elhelyezése, rajzolás. Használják a fordítás funkcióban (integrations.ts).
@pdfme/generator	Sablonok alapján új PDF-eket generál (nem használt a fő folyamatban).

9.2 A pdf-autofill.ts logika

A folyamat négy lépésből áll: (1) PDF bitek letöltése Supabase Storage-ból, (2) AcroForm mezők kinyerése, (3) Profil adatok ratesítése, (4) PDF kimentése.

```
// backend/src/lib/pdf-autofill.ts
export async function analyzePdf(
  form: PdfForm,
  sourcePdf: Uint8Array,
  profile: Partial<Profile>,
  inputValues: Record<string, string> = {},
): Promise<PdfAutofillField[]> {
  const savedFields = normalizeFields(form.mapping)
  // 1. Betöltött mező definíciók az adatbázisból

  const nativeFields = await extractAcroFields(sourcePdf)
  // 2. PDF-ből kinyert AcroForm mezők

  const baseFields =
    savedFields.length > 0
      ? mergeSavedAndNativeFields(savedFields, nativeFields)
      : nativeFields
  // 3. Mentett és natív mezők összefüggése

  const fallbackFields = baseFields.length > 0 ? baseFields : heuristicFields(form)
  // 4. Ha semmi sincs, heurisztikus mezők

  const hydrated = hydrateFieldValues(fallbackFields, profile, inputValues)
  // 5. Értékek behelyettesítése
  return enrichWithTemplateMeta(hydrated, form)
}
```

9.3 A dataKey rendszer -- hogyan csatlakoztunk az adathoz?

Minden PDF mezőnek van egy `dataKey` értéke, amely megmondja honnan kell az adatot venni:

```
// backend/src/lib/pdf-autofill.ts
export function resolveDataKey(
  dataKey: string,
  profile: Partial<Profile>,
  inputValues: Record<string, string>,
): string {
  if (dataKey === "system.today")
    return new Date().toLocaleDateString("ro-RO") // Mai datum

  if (dataKey === "profile.full_address") {
    return [profile.address, profile.city].filter(Boolean).join(", ")
    // "Str. Memorandumului 1, Cluj-Napoca"
  }

  if (dataKey.startsWith("profile.")) {
    const key = dataKey.replace(/^profile\.\/, "") as keyof Profile
    return String(profile[key] ?? "")
    // profile.cnp => profile["cnp"] => "1850312123456"
  }

  if (dataKey.startsWith("input.")) {
    const key = dataKey.replace(/^input\.\/, "")
    return inputValues[key] ?? ""
    // input.make => inputValues["make"] => "Dacia"
  }
}
```

9.4 A guessDataKey heurisztika

Ha egy PDF-ben ismeretlen nevű mező van, a kód megpróbálja kitalálni mit akar:

```
function guessDataKey(label: string): string {
  const value = normalize(label) // Kisbetűs + diakritikus eltávolítás
  if (value.includes("cnp")) return "profile.cnp"
  if (value.includes("email")) return "profile.email"
  if (value.includes("telefon")) return "profile.phone"
  if (value.includes("adresa") || value.includes("domicili"))
    return "profile.full_address"
  if (value.includes("marca")) return "input.make"
  if (value.includes("model")) return "input.model"
  if (value.includes("vin") || value.includes("sasiu"))
    return "input.vin"
  if (value.includes("data")) return "system.today"
  if (value.includes("nume") || value.includes("subsemnat"))
    return "profile.full_name"
  return `input.${safeId(label)}` // Ismeretlen: input-ként kezeli
}
```

9.5 A kitöltés folyamata (fillPdf)

```

export async function fillPdf(
  sourcePdf: Uint8Array,
  fields: PdfAutofillField[],
  profile: Partial<Profile>,
  inputValues: Record<string, string> = {},
): Promise<Buffer> {
  const pdf = await PDFDocument.load(sourcePdf, { ignoreEncryption: true })
  const font = await pdf.embedFont(StandardFonts.Helvetica)
  const hydratedFields = hydrateFieldValues(fields, profile, inputValues)
  const pdfForm = pdf.getForm()

  for (const field of hydratedFields) {
    const value = (field.value ?? "").trim()
    if (!value) continue

    if (field.acroFieldName) {
      try {
        const textField = pdfForm.getTextField(field.acroFieldName)
        textField.setText(latinize(value)) // Roman betűk -> ASCII
        continue
      } catch { /* Ha nem található, rajzolás lesz */ }
    }

    // Rajzolás overlay-ként (ha nincs AcroForm mezo):
    page.drawText(latinize(value), {
      x: field.x, y: height - field.y - field.height + 4,
      size: 10, font, color: rgb(0.05, 0.05, 0.05),
    })
  }
  return Buffer.from(await pdf.save())
}

```

□ Mit tanultam ebből?

- A @pdfme/pdf-lib AcroForm mezokat kinyeri es kitolti PDF-ekbol
- A dataKey rendszer meghatározza honnan jön az adat: profile.*, input.*, system.*
- A guessDataKey heurisztikusan megpróbálja kitalálni az ismeretlen mezo típusat
- A latinize() függvény roman diakritikus jeleket ASCII-ra csere (Helvetica kompatibilis)
- A hydrateFieldValues() összekoti a mezo definiciokat az aktualis adatokkal
- Ha az AcroForm mezo nem taltathato, szoveget rajzol overlay-ként

10. fejezet: Biztonság és audit

Az eCetatean bizalmas személyazonosító adatokat kezel (CNP, lakcím, személyigazolvány szám). Ezért kiemelten fontos a biztonság. Két fő mechanizmus: a hash-lánca audit napló és a parameterizált SQL lekérdezések SQL injection ellen.

10.1 SHA-256 hash függvény

A SHA-256 egy kriptográfiai hash függvény. Fontos tulajdonságai:

- Egyirányú: hash-ból nem lehet visszafejteni az eredetét
- Determinisztikus: ugyanaz az input mindig ugyanazt adja
- Lavina effektus: 1 bit változás teljesen más hash-t ad
- Utókezes mentes (gyakorlatilag): két különböző input azonos hash-t adjon, astronomiailag valószínűtlen

10.2 A hash-chain.ts teljes elemzése

```

// backend/src/lib/hash-chain.ts
import { createHash } from 'crypto'

function sha256(input: string): string {
  return createHash('sha256').update(input).digest('hex')
}

export async function writeAuditEntry({
  userId, action, actionType, data = {}
}: WriteAuditEntryParams): Promise<void> {

  // 1. Legutobb bejegyzes hash-enek lekerese
  const { data: lastEntry } = await supabaseAdmin
    .from('audit_log')
    .select('record_hash')
    .eq('user_id', userId)
    .order('created_at', { ascending: false })
    .limit(1).maybeSingle()

  const previousHash = lastEntry?.record_hash ?? 'GENESIS'
  // Az elso bejegyzes previousHash-e: 'GENESIS'

  const timestamp = new Date().toISOString()

  // 2. Adatok hash-e (az aktualis bejegyzes tartalma)
  const dataHash = sha256(
    JSON.stringify({ action, actionType, timestamp, data })
  )

  // 3. Bejegyzes hash-e = sha256(elozo_hash + adat_hash)
  const recordHash = sha256(previousHash + dataHash)

  // 4. Mentes az adatbazisba
  await supabaseAdmin.from('audit_log').insert({
    user_id: userId, action, action_type: actionType,
    data, data_hash: dataHash,
    previous_hash: previousHash,
    record_hash: recordHash,
  })
}

```

Miert megvaltoztathatatlan? Ha valaki megvaltoztatna az N. bejegyzest, annak data_hash-e megvaltozna, ezert a record_hash is megvaltozna, ami az (N+1). bejegyzes previous_hash-evel nem egyezne. A teljes lanc szervicen vegig leellenorizhető.

10.3 A hash lanc ellenorzese

```
// backend/src/lib/hash-chain.ts
export function verifyHashChain(
  entries: Array<{ data_hash: string; previous_hash: string; record_hash: string }>
): boolean {
  for (let i = 1; i < entries.length; i++) {
    // Az (i). bejegyzes record_hash-enek ujraszamitasa:
    const expectedRecordHash = sha256(
      entries[i - 1].record_hash + entries[i].data_hash
    )

    // Ha nem egyezik: valaki megvaltoztatta!
    if (expectedRecordHash !== entries[i].record_hash) return false

    // Az (i). bejegyzes previous_hash-enek ellenorzése:
    if (entries[i].previous_hash !== entries[i - 1].record_hash) return false
  }
  return true // A lanc intact
}
```

10.4 SQL injection megelőzése

Az SQL injection az egyik legelterjedtebb webes támadás. Ha a felhasználó adatait közvetlenül beszúdjuk az SQL string-be, a támadó SQL kódot illeszt be:

```
// VESZÉLYES (ne csinálj ilyet!):
const query = `SELECT * FROM profiles WHERE cnp = '${userInput}'`
// Ha userInput = "" OR '1'='1", az összes profilt adja vissza!

// BIZTONSÁGOS - parameterizált lekérdezés:
const { data } = await supabaseAdmin
  .from('profiles')
  .select('user_id')
  .eq('cnp', cnp) // A Supabase SDK automatikusan parameterizál
// Ez: WHERE cnp = $1 (a $1 értéket biztonságosan aadja át)
```

A Supabase JS kliens MINDIG parameterizált lekérdezéseket használ. Emiatt az eCetatean-ban nincsen SQL injection sebezhetőség.

10.5 CORS (Cross-Origin Resource Sharing)

A böngésző biztonsági szabályai tiltják, hogy az egyik domain JavaScript-je keresse küldjön másik domain API-jának. Ezért kell CORS konfigurálást:

```
// backend/src/middleware/cors.ts (valoszinusitheto tartalom)
import { cors } from 'hono/cors'

export const corsMiddleware = cors({
  origin: process.env.FRONTEND_URL || 'http://localhost:5173',
  // Csak errol a domainrol engedi a kereseket
  credentials: true, // Cookie-k engedelyezese
  allowMethods: ['GET', 'POST', 'PATCH', 'DELETE', 'OPTIONS'],
})

// Ha a frontend https://app.ecetatean.ro domain-rol
// probalna elerni az API-t, es CORS nincs beallitva,
// a bongeszo blokkolna a keres. Ezert kell explicit engedely.
```

☐ Mit tanultam ebbol?

- ☐ A SHA-256 egyiranyu hash: hash-bol nem fejtheto vissza az eredeti adat
- ☐ A hash lanc (blockchain-szeru) megvaltoztathatatlanra teszi az audit naplot
- ☐ A verifyHashChain ellenorzi, hogy az osszes bejegyzes hash-e konzisztens-e
- ☐ A SQL injection parameterizalt lekerdesezsekkel megelozhető (Supabase SDK csinalja)
- ☐ A CORS ellenorzi, hogy melyik domain kuldhethet HTTP kereseket az API-hoz
- ☐ A service_role Supabase kulcs SOHASEM kerul a frontendre (csak backenden)

11. fejezet: Deployment

A 'deployment' azt jelenti: a helyi gépen írt kód elérhető lesz az interneten bárki számára. Az eCetatean két különböző platformot használ a két részhez.

11.1 Cloudflare Workers: edge computing

A frontend Cloudflare Workers-en fut. Ez nem egy hagyományos szerver -- Cloudflare 300+ globális adatközpontjában fut a kód, mindig a felhasználóhoz legközelebbi szerverről szolgáltatva.

```
{
  // frontend/wrangler.jsonc
  "$schema": "node_modules/wrangler/config-schema.json",
  "name": "tanstack-start-app",
  "compatibility_date": "2025-09-24",
  "compatibility_flags": ["nodejs_compat"],
  "main": "src/server.ts"
}

// Deployment parancs:
// npx wrangler deploy
// Ez feltölti a build kimerete Cloudflare-re és globálisan elérhető lesz
```

Az `nodejs\_compat` flag lehetővé teszi a Node.js API-k használatát (pl. crypto, Buffer) a Worker környezetben, ahol alapból csak Web API-k érhetők el.

11.2 Vercel: serverless functions

A backend Vercel-en fut. Minden API kérésre egy új serverless function példányt indít. Nincsen állandóan futó szerver -- a Vercel 'on demand' indítja a kódot.


```
{
  // backend/vercel.json
  "version": 2,
  "builds": [
    {
      "src": "src/index.ts",
      "use": "@vercel/node"
      // @vercel/node lefordítja és futtatja a TypeScript kódot
    }
  ],
  "routes": [
    {
      "src": "/api/(.*)", // Minden /api/... URL...
      "dest": "src/index.ts" // ...az index.ts-hez megy
    },
    {
      "src": "/(.*)", // Minden más URL is
      "dest": "src/index.ts"
    }
  ],
  "env": {
    "SUPABASE_URL": "@supabase_url",
    // @supabase_url = Vercel secret neve (nem a valódi érték!)
    "SUPABASE_SERVICE_ROLE_KEY": "@supabase_service_role_key",
    "SUPABASE_JWT_SECRET": "@supabase_jwt_secret",
    "GEMINI_API_KEY": "@gemini_api_key",
    "FRONTEND_URL": "@frontend_url"
  }
}
```

11.3 Környezeti változók (Environment Variables)

Sohasem szabad titkos kulcsokat (API kulcsok, adatbázis jelszavak) a forráskódba írni! Ezért léteznek a környezeti változók:

```
# backend/.env.local (NEM kerül git-be! .gitignore-ba van!)
SUPABASE_URL=https://xxxxx.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIs...
SUPABASE_JWT_SECRET=super-titkos-jwt-secret
ANTHROPIC_API_KEY=sk-ant-...
GEMINI_API_KEY=AIza...
AUTH_STUB=true # Fejlesztésben: átugorja a JWT-t
AUTH_STUB_ROLE=citizen # Stub felhasználó szerepe

# frontend/.env.local
VITE_API_URL=http://localhost:3001 # Backend URL
VITE_SUPABASE_URL=https://xxxxx.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=eyJhbGciOi... # Anon kulcs (publikus)
```

A `VITE\_` előtagg fontos: csak ezek a változók kerülnek bele a frontend build-be. A VITE_ nélküli változók biztonságban maradnak a szerveren.

11.4 Az AUTH_STUB pattern értéke

Az `AUTH\_STUB=true` egy elegans fejlesztési mintázat: lehetővé teszi a backend API teszteket bejelentkezési folyamat nélkül. Produkciós környezetben sohasem szabad bekapcsolni (a Vercel környezeti változókban ez az érték nincs beállítva).

☐ **Mit tanultam ebből?**

- ☐ A Cloudflare Workers edge computing: 300+ helyszínen fut, minimalis latencia
- ☐ A Vercel serverless functions 'on demand' indítják a kódot minden keresésre
- ☐ A wrangler.jsonc konfigurálja a Cloudflare deploymentet (name, compat_date, main)
- ☐ A vercel.json a route-okat és a build konfigurációt határozza meg
- ☐ A környezeti változók titkok tárolására szolgálnak (sohasem a forráskódoba!)
- ☐ Az AUTH_STUB=true fejlesztési gyorsbillentyű -- sohasem produkcióba!

12. fejezet: Összefoglalás és következő lépések

Gratulálok! Vegigmentel az eCetatean projekt összes fontos technológiáin. Ez a fejezet összefoglalja a kapcsolatokat a technológiák között, és megmutatja hova érdemes továbbmenni.

12.1 Technológia térkép

Technológia	Hol van a projektben	Egyértelmű cél
TypeScript	Mindenhol (frontend + backend)	Tipusbiztos, kevesebb bug
React 19	frontend/src/routes/*.tsx	UI komponens rendszer
TanStack Router	frontend/src/routes/ fájlok	Fájl-alapu URL routing
TanStack Query	useProfile(), useNews() stb.	API adat cacheles
Tailwind CSS	className='...' minden jsx-ben	Gyors, egységes stílus
Framer Motion	motion.div, slideUpSheet stb.	Animációk
Hono.js	backend/src/index.ts	HTTP szerver framework
Supabase	supabase.ts + SQL lekérdesek	Adatbázis + Auth + Storage
JWT	supabase-jwt.ts + middleware/auth.ts	Autentikáció
Claude/Gemini	claude-claudia.ts + gemini-claudia.ts	AI asszisztens
pdf-lib	pdf-autofill.ts	PDF kitöltés
Cloudflare	wrangler.jsonc	Frontend hosting (edge)
Vercel	vercel.json	Backend hosting (serverless)

12.2 Hogyan kapcsolódnak a technológiák más projektekhez?

Az eCetatean-ban tanult dolgok szorosan kapcsolódnak a más népszerű projektekhez:

- Hono.js -> Express.js: Az Express a legelterjedtebb Node.js framework. Szinte ugyanolyan API, de Hono gyorsabb és TypeScript-natív.
- Supabase -> Prisma + raw PostgreSQL: Prisma egy TypeScript ORM, ami szintén SQL felett működik. A Supabase SDK és a Prisma hasonló koncepteket használ.
- TanStack Router -> Next.js App Router: A Next.js 13+ hasonló fájl-alapu routingot használ. Mindket rendszer server-side rendering-et támogat.
- TanStack Query -> SWR, React Query: Ugyanaz az alap idea: API adat cacheles React hook-okkal.
- Vercel serverless -> AWS Lambda, Google Cloud Functions: Ugyanaz a paradigma: kód fut 'on demand', nem állandóan futó szerver.
- Cloudflare Workers -> Deno Deploy, Fastly Compute@Edge: Edge computing alternatívák.

12.3 Tanulási útmutató

A következő sorrend ajánlott:

Lépcsők	Mit tanulj	Forrás
---------	------------	--------

1. Alap	HTML, CSS, JavaScript (vanillia)	MDN Web Docs (mdn.dev)
2. TypeScript	Tipusok, interface-k, generikusok	typescriptlang.org
3. React	Komponensek, hooks, state	react.dev (hivatalos)
4. Node.js	Event loop, npm, modul rendszer	nodejs.org
5. SQL	SELECT, JOIN, INDEX, transaction	PostgreSQL tutorial
6. HTTP/REST	Kerelem-valasz, statusz kodok, headers	REST API tutorial
7. JWT/Auth	Token struktura, ellenorzes, refresh	jwt.io
8. Deployment	Docker, CI/CD, cloud platformok	Vercel/Cloudflare docs

12.4 Hivatkozások és dokumentációk

- React 19: react.dev
- TanStack Router: tanstack.com/router
- TanStack Query: tanstack.com/query
- Tailwind CSS: tailwindcss.com
- Framer Motion: motion.dev
- shadcn/ui: ui.shadcn.com
- Hono.js: hono.dev
- Supabase: supabase.com/docs
- PostgreSQL: postgresql.org/docs
- Anthropic Claude: docs.anthropic.com
- Google Gemini: ai.google.dev
- Vercel: vercel.com/docs
- Cloudflare Workers: developers.cloudflare.com
- TypeScript: typescriptlang.org/docs
- pdf-lib: pdf-lib.js.org

☐ **Vegso osszefoglalas: Mit tanultam ebbol az utmutatobol?**

- ☐ A modern webalkalmazasok haromretegUek: frontend (React), backend (Hono.js), adatbazis (Supabase)
- ☐ A TypeScript statikus tipusossag forditasi hibakat talal, javitja a kod minosegat
- ☐ A React komponens-alapu UI rendszer: useState, useEffect, custom hooks
- ☐ A Tailwind utility-first CSS: gyors UI fejlesztas CSS fajok nelkul
- ☐ A Hono.js REST API szerver: route-ok, middleware, request/response ciklus
- ☐ A PostgreSQL relationalas adatbazis: tablak, indexek, RLS biztonsag
- ☐ A JWT autentikacio: header.payload.signature, ellenorzes a backenden
- ☐ Az LLM API (Claude/Gemini) tool use-sal: AI eszkoz hivasok, streaming
- ☐ A PDF autofill: AcroForm mezok, dataKey rendszer, latinize
- ☐ A hash lanc audit naplo: SHA-256, megvaltoztathatatlan bejegyzesek
- ☐ A deployment: Cloudflare Workers (edge) + Vercel (serverless)
- ☐ Kornyezeti valtozok: titkos kulcsok sose a forraskodoba!